

# IVB Model — Optimized

Initial Balance breakout with volume-profile location & order-flow confirmation  
Design, trade logic, module contracts, inputs & outputs, vectorized internals, caching & timing · package `strategies/ivb_model_optimized/` · updated 2026-07

## 0. What the strategy is

The IVB (Initial-balance Volume-profile Breakout) model is a directional intraday futures strategy. Its thesis: the market spends the first part of the session discovering a fair-value range (the initial balance); when price finally commits beyond that range it reveals who won the day's most important auction; the highest-probability entry is not the breakout itself but the first pullback that fails — where price returns toward value, the counter-move runs out of aggressive participants, and the order flow shows that failure directly. Originally inspired by Fabio Valentini's IVB concept, rebuilt as a modular plug-in package.

The design has two fixed layers and two pluggable ones. The **directional framework** (define the initial balance, map the value area, detect a breakout, wait for a retest) is fixed. The **entry-confirmation layer** is pluggable — seven interchangeable order-flow patterns, each switchable independently. The **risk layer** is pluggable — four interchangeable stop/target scripts. Everything that differs between research variants lives in those two registries; the spine never changes.

| Layer            | The question it answers                                      | How the model answers it                                                              |
|------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1 · Direction    | Who won the most important battle of the day?                | The first post-IB bar to CLOSE beyond the initial-balance high (long) or low (short). |
| 2 · Location     | Where is the reload / value zone the pullback must fail at?  | VAH / POC / VAL, built from the IB's traded-volume-at-price profile.                  |
| 3 · Confirmation | Is the counter-move actually failing at that zone right now? | One of seven order-flow entry patterns (Section 5) fires inside the value area.       |

**How to read this document.** Sections 2–8 explain the trade logic with the exact numeric rules the code enforces. Sections 9–12 document what is new in the optimized package: the vectorized internals, the day-core cache, the timing instrumentation and the measured performance. Sections 13–17 are the contract: parameters, outputs, data requirements, the equivalence guarantee and known limits. The companion `strategies/ivb_model_optimized/CLAUDE.md` is the terse code-map for engineers.

## 1. This package vs. the original ivb\_model

`ivb_model_optimized` is a performance rewrite of `strategies/ivb_model/`. **The trade logic, every parameter, and every output are identical** — verified byte-for-byte (including the `notes` JSON strings) against the original over the full 307-day ES dataset across 12 parameter configurations covering all four risk scripts, all seven entries, both VWAP TP modes, late/in-profit trailing and the timeout tails (Section 16). What changed is only *how* the same numbers are computed:

- **Vectorized day context** — each day is materialized once as numpy arrays; the JSON columns are parsed once per day instead of per finder per bar; windows are positional views, not DataFrame slices (Section 10).
- **Day-core cache** — the parsed, param-independent per-day context is kept in RAM across backtest runs, so re-running with different parameters skips file reads and parsing entirely (Section 11).
- **Gated per-run state** — baselines are built only for the finders/detectors actually enabled by the bit strings.
- **Stage timers** — every pipeline stage is timed; one aggregated table prints per run (Section 12).
- **Pruned, prefetched I/O** — only consumed parquet columns are read and the next days prefetch on a small thread pool.

Measured on 307 ES days, everything enabled: original **26.8 s** → first (cold) run **3.9 s** → every following (warm) run **0.7 s**. Full breakdown in Section 12.

## 2. Package structure & file contracts

A multi-file Python package (folder with `__init__.py`). The backtester discovers it by folder name and calls `run(folder_path, start_date, end_date, params) -> pd.DataFrame`, exactly like a one-file strategy. It also reads `PARAMS` and `PARAM_SECTIONS` for the UI. Internally:

| File                       | Provides                                                                                | Role / contract                                                                                                                                                                                                                     |
|----------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>__init__.py</code>   | <code>run(...), PARAMS, PARAM_SECTIONS</code>                                           | Globs day files in range, resolves the indicators folder, runs the day-core cache + prefetch pipeline, calls <code>process_day</code> per day, prints the cache line + timing table, returns the <code>OUTPUT_COLUMNS</code> frame. |
| <code>params.py</code>     | <code>PARAMS, PARAM_SECTIONS, OUTPUT_COLUMNS</code>                                     | Default parameter dict, UI grouping, output schema.                                                                                                                                                                                 |
| <code>core.py</code>       | <code>build_day_core, detect_breakout, detect_retest, find_entry, process_day</code>    | Builds the cacheable day context; orchestrates one day on absolute bar positions; dispatches entry finders and the risk script.                                                                                                     |
| <code>_daydata.py</code>   | <code>DayData, EntryWindow, TradeWindow, parse_json_column, prev_rolling_max/min</code> | The per-day numpy context and the shared window masks (Section 10).                                                                                                                                                                 |
| <code>_timing.py</code>    | <code>timed, reset, report</code>                                                       | Accumulating stage timers; one printed table per run (Section 12).                                                                                                                                                                  |
| <code>profile.py</code>    | <code>compute_ivb_profile(day, ib_end)</code>                                           | Peak-based volume profile -> (poc, vah, val) from pre-parsed tick_volume (Section 6).                                                                                                                                               |
| <code>baselines.py</code>  | <code>build_rolling_baseline, build_passive_baseline, build_cvd_change_baseline</code>  | The three day-level rolling baselines, returned as arrays aligned to the session (Section 4).                                                                                                                                       |
| <code>absorption.py</code> | <code>absorption_scan, wick_bounds</code>                                               | Shared absorption level scan on pre-parsed tick_volume; first qualifying level in JSON document order (identical to the old dict-iteration order).                                                                                  |
| <code>entries/</code>      | 7 finders + <code>FINDER_REGISTRY</code> + <code>FINDER_NAMES</code>                    | One module per pattern; each exposes <code>find_entry(win, params)</code> returning a 5-tuple of window-relative indices (Section 5).                                                                                               |
| <code>risk/</code>         | 4 scripts + <code>RISK_REGISTRY</code> + <code>RISK_NAMES</code>                        | Self-contained stop/target placement + fill simulation; selected 1-based by <code>risk_script</code> (Section 7).                                                                                                                   |

## Per-day call chain

```

run() -> for each YYYY-MM-DD.parquet in [start, end]:
    key = (candle path, mtime, size) + (indicator path, mtime, size)
    day = _DAY_CACHE[key]                # HIT: skip everything below
    on MISS:
        read candle parquet (9 columns) + indicator parquet (9 of ~32 columns) [prefetch thread]
        build_day_core(session, ind_df) -> DayData (arrays, lazy JSON, cvd_raw, bands_raw)
        _DAY_CACHE[key] = day            # LRU, 600-day cap

process_day(day, params):
    0. day.reset_run_state()              # wipe the previous run's baselines
    1. need >= ib_minutes RTH bars (09:30-16:00 NY)
    2. IB high/low from first ib_minutes bars (abort if range <= 0)
    3. detect_breakout(post-IB closes) -> direction, breakout_pos (abort if none)
    4. compute_ivb_profile(IB bars) -> poc, vah, val [profile.py]
    5. GATED: rolling/passive baselines, CVD std, VWAP bands - only what the
       enabled bits consume (Section 4) [baselines.py]
    6. loop (<= max_flips):
        detect_retest()                  -> retest_pos (else: no trade)
        post_retest = EntryWindow([breakout bar] + [retest .. retest+entry_window])
        find_entry() -> earliest entry across enabled finders wins
        entry -> break out of loop
        invalidate -> flip direction, resume AT the invalidation bar, re-detect
    7. RISK_REGISTRY[risk_script-1](entry_win, trade_win, ...) -> trade dict | None
    8. attach trade_type + notes (JSON)

```

### 3. The directional framework (the spine)

Deterministic and identical for every entry and risk plug-in. It turns a day of candles into a direction, a value area to fade toward, and a bounded window in which the confirmation layer may look for a trade.

**Step 1 — Initial Balance.** Take the first `ib_minutes` bars of the regular session (default 30 → 09:30–09:59 NY). IB high = max of those highs, IB low = min of those lows. If the range is zero the day is skipped. The bar immediately after the IB is the first candidate breakout bar.

**Step 2 — Value area (location).** From the same IB bars the model builds a volume-at-price profile from `tick_volume` and extracts the POC (most-traded price) and the VAH / VAL (top and bottom of the 70% value area). Computed once per day, reused across every flip. Algorithm in Section 6.

**Step 3 — Breakout (direction).** Scan post-IB bars for the first CLOSE beyond a wall: close > IB-high → long, close < IB-low → short; on a same-bar tie, long wins. A wick that pokes through but closes back inside is not a breakout — the committed close filters stop-runs.

**Step 4 — Retest.** After the breakout, wait up to `retest_window` bars for a pullback into value: long needs a bar low <= VAH, short a bar high >= VAL. The window handed to the confirmation layer (`post_retest`) is the breakout bar (kept as index 0 for reference) followed by the retest bar and the next `entry_window` bars. No retest in time → no trade.

**Step 5 — Invalidation & direction flipping.** The idea dies when price closes back through the far side of value: long on close < VAL, short on close > VAH. A decisive failure is treated as information: the model flips direction, resumes scanning AT the invalidation bar, and re-detects a breakout in the new direction — up to `max_flips` times (default 4), always reusing the same IB profile. If the re-detected breakout does not point the flipped way, the day is dropped rather than forced.

**Why fade toward value instead of chasing the break?** A real breakout gets defended on the retest: the players who pushed price out of balance step back in to stop it re-entering. That defense is visible in order flow as absorption, resting size, or a delta divergence. Fading the failing pullback gives a tight, logical stop (the value-area edge) and a clear invalidation, instead of buying strength at the worst price.

### 4. Absorption grading, the rolling baselines & gating

"Absorption" is the core order-flow primitive: aggressive traders keep hitting a level, yet price refuses to move through it — someone is quietly filling all of that aggression with passive orders. Four of the seven finders lean on the same grading so the definition stays consistent.

#### What makes a candle an absorption candle

A bar qualifies when BOTH hold on the defended side (lower wick for a long, upper wick for a short):

- **Shape** — the defended wick is at least `wick_threshold` of the bar's total range (default 40%): price was pushed into the zone and rejected, leaving a long tail. (Vectorized as the per-bar `wick_frac` array on the window context; bars with range <= 0 get -inf and can never qualify.)
- **Flow** — somewhere inside that wick a single price level shows counter-trend aggressive volume >= baseline × `absorption_mult` (default 2×). Long: aggressive selling soaked up at the lows; short: aggressive buying soaked up at the highs.

`absorption_scan(tv, wick_low, wick_high, required, direction)` performs the level scan on the pre-parsed `tick_volume` arrays and returns (`found, trigger_price, trigger_volume`) for the FIRST qualifying level in JSON document order — the same level the original dict-iteration loop found; the trigger is written into the trade notes. The multiplier and wick threshold are per-finder (the consecutive and passive finders pass their own values), so absorption can be graded more or less strictly per pattern without changing the shared routine.

#### The rolling baselines (what "big" is measured against)

Raw volume is meaningless without a local reference. Each baseline is a rolling day-level average over `absorption_baseline_window` bars (default 20), and all of them skip the first five minutes of RTH (baselining starts 09:35) because the opening bars are abnormally heavy. During warm-up the baseline is NaN, so the earliest candles simply cannot be graded. The pandas `rolling(window, min_periods=window)` pipelines are kept verbatim from the original so the floats stay bit-identical; results are returned as numpy arrays positionally aligned to the session.

| Baseline                               | What it averages                                                                                                                                                                                                                                                                                                   | Used by                                          |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <code>build_rolling_baseline</code>    | Per-tick aggressive volume = buy/sell volume / bar range in ticks. Returns a sell baseline (graded by longs) and a buy baseline (shorts).                                                                                                                                                                          | absorption_delta, consecutive, passive size-only |
| <code>build_passive_baseline</code>    | The largest resting order size on the defended side per bar (bids below the open for longs, asks above it for shorts). Sparse — the rolling window slides over bars WITH valid passive data only. The per-bar maxima come from one vectorized pass over the day's concatenated levels, memoized on the cached day. | passive size-only, passive wall                  |
| <code>build_cvd_change_baseline</code> | Rolling standard deviation of bar-to-bar CVD change — the session's own order-flow volatility, used to z-score divergences.                                                                                                                                                                                        | both CVD-divergence finders                      |

A fourth, finder-local baseline lives inside the two-bar modules: the strided pair-density baseline averages the defended-side per-tick volume of merged two-bar pairs preceding the pattern (that finder grades a synthetic two-candle unit). It is precomputed once per window as a density array (`density[j]` of pair (`j`, `j+1`)) and gathered per candidate —  $O(n)$  instead of the original  $O(n^2)$  rebuild — summing in the original loop order so the floats match exactly.

### Gating — only what is consumed gets built

Baselines are param-dependent, so they are rebuilt every run — but only when something enabled actually reads them. The gate uses exactly the bit strings the dispatchers use (`valid_entries`, unioned with `trailing_entries` when `risk_script = 4`):

| Per-run state                    | Built when                                                                      |
|----------------------------------|---------------------------------------------------------------------------------|
| rolling sell/buy baselines       | finder bit 1, 2 or 4 is on (absorption_delta / consecutive / passive size-only) |
| passive baselines (long + short) | finder bit 4 or 5 is on (passive size-only / passive wall)                      |
| CVD + CVD-change std             | finder bit 6 or 7 is on AND the day has indicators                              |
| VWAP band attachment             | <code>risk_script</code> is 3 or 4                                              |

A baseline left None is never read: the window contexts skip gathering it, and every consumer is switched off by the same bits that gated it.

## 5. The seven entry finders

Every finder shares one signature: `find_entry(win, params)` where `win` is the shared `EntryWindow` context, returning (`entry_rel`, `entry_price`, `invalidation_rel`, `entry_notes`, `trade_type`) with bar indices relative to the window (the dispatcher maps them back to positions and timestamps; ascending index equals ascending time). The dispatcher (`find_entry` in core) runs every finder switched on by the `valid_entries` flag string over the identical window and shared masks, then picks the **earliest entry**. If no finder entered but one signalled invalidation, the **earliest invalidation** is returned and drives a flip. Universal rules:

- **Entry price is always the OPEN of the bar after the confirming candle** — the signal must fully close before the model commits; no intrabar look-ahead.
- **Whole-trade invalidation short-circuits every finder**: a close through VAL (long) / VAH (short) at or before a candidate bar returns an invalidation instead of an entry. Precomputed once per window as the `invalid` mask / `first_inv` index.
- The breakout bar (window index 0) can invalidate but can never be a signal candle.

The `valid_entries` string is one bit per finder in registry order (1 = on). The default `"1111100"` enables the five order-flow patterns and leaves the two CVD-divergence finders off (they need the optional indicators dataset). Bit order is fixed:

| # | trade_type | The pattern, in words | Key params |
|---|------------|-----------------------|------------|
|---|------------|-----------------------|------------|

|   |                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                         |
|---|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>absorption_delta</code>             | An absorption candle at the zone, then a confirming candle with body $\geq$ threshold and delta pushed past $\pm$ threshold. The classic "absorb then reverse". A later close back through the absorption level (long: strictly below the signal bar's low) kills that candidate before confirmation. NOTE: this finder's confirming candle does NOT require the candle colour — body + delta suffice (all other confirm scans also require the correct colour).                           | <code>wick_threshold,</code><br><code>absorption_mult,</code><br><code>delta_threshold,</code> <code>body_threshold</code>                              |
| 2 | <code>consecutive_absorption</code>       | N absorption candles (default 2) clustering within a few ticks of the same level AND body midpoint — repeated defense of one price. No delta confirmation; the repetition is the signal; enters on the next bar's open. A level price has since closed through is dead (long: any later close strictly below it; closing exactly AT it keeps it).                                                                                                                                          | <code>consec_abs_n,</code> <code>consec_abs_mult,</code><br><code>consec_abs_ticks,</code><br><code>consec_wick_threshold</code>                        |
| 3 | <code>two_bar_absorption</code>           | A tight reversal pair — bar 1 against the direction, bar 2 with it, both with small defended wicks ( $\leq$ <code>two_bar_wick_ticks</code> ), bar 2 with just-positive (long) / just-negative (short) delta — merged into one synthetic candle (summed tick_volume), graded against the two-bar pair baseline, absorption must register in the defended half of the merged candle, then a confirming candle scanned from bar 2 inclusive. Catches absorption that splits across two bars. | <code>two_bar_wick_ticks,</code><br><code>two_bar_abs_mult</code>                                                                                       |
| 4 | <code>passive_absorption_size_only</code> | A genuinely large resting order on the defended side (raw size $\geq$ passive baseline $\times$ mult, order count $> 0$ , price below the open for longs / above for shorts) AND absorption on the same candle, then a confirming candle. Passive intent + realised absorption together. The notes record the FIRST qualifying resting level.                                                                                                                                              | <code>passive_size_order_mult,</code><br><code>passive_size_absorption_mult,</code><br><code>passive_size_wick_threshold</code>                         |
| 5 | <code>passive_wall</code>                 | A cluster of big resting orders (default 3, raw size $\geq$ baseline $\times$ mult) stacked within <code>passive_wall_ticks</code> of one level, accumulated across bars (the running list is never cleared). No absorption candle and no delta — the wall of stacked liquidity is the signal; enters on the bar after the wall completes.                                                                                                                                                 | <code>passive_wall_n,</code><br><code>passive_wall_mult,</code><br><code>passive_wall_ticks</code>                                                      |
| 6 | <code>cvd_divergence_absorption</code>    | A CVD divergence read as absorption: the newer pivot could NOT make a new extreme (lower-or-equal high for shorts / higher-or-equal low for longs, within a tick tolerance) even though cumulative delta pushed further into it — the extreme was absorbed. Needs CVD; z-scored against the session's CVD-change volatility.                                                                                                                                                               | <code>cvd_pivot_k,</code> <code>cvd_min/</code><br><code>max_separation,</code><br><code>cvd_wick_tolerance_ticks,</code><br><code>cvd_min_score</code> |

|   |                                        |                                                                                                                                                                                                                                           |                                                                                                           |
|---|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 7 | <code>cvd_divergence_exhaustion</code> | The mirror, read as exhaustion: price DID make a new extreme but cumulative delta failed to follow — the move ran out of fuel. Needs CVD. Deliberately independent: its own module copy and its own <code>cvd_exh_*</code> parameter set. | <code>cvd_exh_pivot_k, cvd_exh_min/max_separation, cvd_exh_wick_tolerance_ticks, cvd_exh_min_score</code> |
|---|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|

## The confirming candle (patterns 1, 3, 4, 6, 7)

Most finders separate detection from confirmation. After spotting absorption or a divergence they scan up to `entry_after_absorption` bars (default 5) for a confirming candle: correct colour (except pattern 1 — see table), `body >= body_threshold` of range, and signed `volume_delta_pct` past  $\pm \text{delta\_threshold}$ . Only then — on the next bar's open — is the position taken. Patterns 2 and 5 are self-confirming and enter on the bar right after the pattern completes. Both confirm tests are precomputed once per window as boolean masks (`confirm_any` without the colour test, `confirm_dir` with it) and shared by every finder and every trailing detector.

## How the two CVD-divergence finders work

Both build swing pivots with a left-side k-bar fractal (bar i-1 is a candidate when its extreme strictly exceeds the previous k bars') confirmed fast by a single reversal candle at bar i (no waiting k bars right). The true pivot extreme is the more-extreme of the candidate and its confirming bar (ties go to the candidate). The two most-recent confirmed pivots are graded: separation within `[min_separation, max_separation]`, the price condition per flavour (with a wick tolerance in ticks), and the CVD gap turned into a z-score by dividing by the CVD-change std at the newer pivot — it must exceed `cvd_min_score` (default 0.3), so "divergence" is measured relative to how noisy that day's order flow is. A freshly confirmed third pivot supersedes any pending setup; a pending setup is abandoned when the confirm window (`entry_after_absorption`) elapses. If no indicators were loaded, both finders quietly disable themselves and return all-None, never blocking the others.

## Adding an eighth entry type

Drop `entries/my_entry.py` exposing `find_entry(win, params) -> 5-tuple` (window-relative indices); append it to `FINDER_REGISTRY` and its name to `FINDER_NAMES`; extend the default `valid_entries` string by one bit and add any new parameters to `params.py`. The `EntryWindow` gives you the arrays and masks (Section 10) — no DataFrame access needed.

## 6. Volume-profile algorithm (peak-based)

A naive "most-traded price" POC is fragile — one fat tick can move it. IVB uses a peak-based method so the value area sits on a genuine cluster of activity:

- Aggregate tick\_volume across all IB bars into a {price: total\_volume} map (from the pre-parsed arrays; python-int accumulation in document order, preserving the original's exact sums and tie-breaks).
- Smooth with a 3-tick rolling average to remove single-tick spikes, then find local maxima.
- Cluster peaks within 4 ticks of each other, keep the strongest per cluster, take the top 5.
- Refine each surviving peak to the true highest-volume raw tick within  $\pm 3$  ticks → a POC candidate.
- Expand a 70% value area outward from each candidate; the candidate with the tightest value area (smallest VAH-VAL) wins — concentration beats sprawl.
- Re-pin the POC to the highest raw-volume tick inside the winning value area.

If fewer than three distinct price levels exist (a very thin IB), it falls back to a simple max-volume POC expanded to 70%. The peak / VA-expansion logic is kept verbatim from the original (it is  $O(\#levels)$  and floating-point-order sensitive).

## 7. Risk management — four interchangeable scripts

`risk_script` is a 1-based selector into `RISK_REGISTRY` (out-of-range falls back to 1). Each script is fully self-contained: it owns its stop/target placement AND its own copy of the fill simulator — no shared module, no cross-script imports, so any script can be edited in isolation. All four share the signature `run(entry_win, trade_win, entry_pos, entry_price, direction, levels, params)` — `entry_win` is the post-retest `EntryWindow` (for pullback stops), `trade_win` the post-entry `TradeWindow`, `entry_pos` the absolute day position of the entry bar; `levels`

carries val/vah/poc and the day context rides on `trade_win.day` (baselines, CVD, VWAP band arrays). Each returns the standard trade dict or None.

| # | Script                          | Stop placement                                                                                                                                                                                                            | Target                                                                                                                                   |
|---|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>basic_risk</code>         | VAL/VAH ( <code>sl_type=0</code> ) or the swing low/high of the post-retest bars up to and including the entry bar ( <code>sl_type=1</code> )                                                                             | Fixed reward multiple: entry $\pm$ risk $\times$ <code>rr</code>                                                                         |
| 2 | <code>zone_sl_risk</code>       | Where the pullback actually bottomed/topped by CLOSE, snapped to the VAL/POC/VAH zone it landed in — or the actual wick extreme if price poked beyond the zone edge. Breakout bar and entry bar excluded (no look-ahead). | Fixed reward multiple: entry $\pm$ risk $\times$ <code>zone_rr</code>                                                                    |
| 3 | <code>vwap_tp_risk</code>       | VAL/VAH or the zone logic, chosen by <code>sl_placement</code> ; fixed for the whole trade                                                                                                                                | A tick-VWAP $\pm 2$ -sigma / $\pm 3$ -sigma deviation band (globex or RTH session), frozen at entry ("now") or trailed live ("trailing") |
| 4 | <code>vwap_trailing_risk</code> | As script 3, PLUS a signal-driven trailing stop: the seven entry-style signals are re-detected on the live trade bars and ratchet the stop (details below)                                                                | As script 3 (same escalation and 1:1 fallback)                                                                                           |

### VWAP target selection & escalation (scripts 3 and 4)

The target band is `vwap_tick_{session}_std{2|3}_{up|dn}` picked by `vwap_session`, `vwap_std` and the trade direction. If at entry price already sits past the chosen band, the script escalates: past 2-sigma bumps the target to 3-sigma; past 3-sigma falls back to a plain 1:1 target. In "trailing" mode the band target moves bar-by-bar with a PESSIMISTIC same-bar tie (the stop wins; NaN band bars can only trigger the stop); in "now" mode the band value is frozen at entry. **Both scripts require the bands:** if the day has no indicators / band columns, or the band is NaN at entry, they return None — no trade. `risk_notes` records `tp_type` (`tp_vwap_2` / `tp_vwap_3` / 1:1) and `escalated`.

### Script 4 — the signal-driven trailing stop

While the trade is open, up to seven trail detectors re-detect the entry-style signals on the live bars, gated by the `trailing_entries` bit string (same order as `valid_entries`). Differences from the entry finders: there is NO VAL/VAH invalidation in-trade (the stop manages the exit); EVERY signal — including consecutive-absorption and passive-wall, which enter without one — requires a confirming candle (correct colour + body + delta); and every occurrence is collected, not just the first. A signal confirmed on bar *j* proposes its candidate stop (the signal candle's defended extreme: low for longs, high for shorts; the merged-candle extreme for two-bar; the 2nd pivot price for CVD) which takes effect from bar *j*+1. The stop only ever tightens.

- `trailing_in_profit` (default 1) — 1: a signal whose candidate stop is still in loss (below entry for longs / above for shorts) is not even logged; 0: every signal is logged and may trail, loss or not.
- `late_trailing` (default 0) — 1: each logged signal trails the stop to the PREVIOUS logged signal's level (the first logged signal only arms the log); 0: a signal trails to its own level immediately.

A stop hit at a trailed level reports `exit_reason = "trailing_sl"` and exits at the trailed level, while the `sl` output column always keeps the originally placed stop. Applied trails are recorded in `risk_notes: trail_count` plus flat `trailN_time` / `trailN_type` / `trailN_stop` keys per applied trail, carrying the same fields that finder's entry notes would carry (absorption/trigger/passive/wall/CVD-pivot details), plus `trailN_trigger_type` / `trailN_trigger_time` for late trails. Only trails in effect by the exit bar are reported.

### Trade fill simulation (shared three-phase shape)

Once stop and target are set, every script simulates the fill the same way (script 4 uses the per-bar trailing stop series in place of a constant stop):

- **Phase 1 — up to `trade_timeout` bars:** walk forward for stop or target. With a fixed target, the target wins a same-bar tie (optimistic); in band-trailing mode the tie is pessimistic (stop wins). Exit reason `tp / sl / trailing_sl`.
- **Phase 2 — the timeout bar:** in profit at the timeout close → bank it (`tp_timeout`). Otherwise pull the target to breakeven and the stop to the swing extreme of the timeout window — in script 4 the swing stop never loosens the trailing ratchet (the tighter of the two applies per bar).
- **Phase 3 — after the timeout:** exit on the breakeven target (`tp_timeout`), the adjusted stop (`sl_timeout` / `trailing_sl`), or the session close (`eod`).

The default `trade_timeout = 999` exceeds a normal RTH session (~390 one-minute bars), so with defaults a trade almost always resolves in Phase 1 and the breakeven-and-run machinery stays dormant. Lower the value to activate it.

| exit_reason              | Meaning                                                                                           |
|--------------------------|---------------------------------------------------------------------------------------------------|
| <code>tp / sl</code>     | Target / stop touched within the timeout window.                                                  |
| <code>trailing_sl</code> | (script 4) The stop that was hit had been ratcheted by a trail signal; exit at the trailed level. |
| <code>eod</code>         | Session ended before any level was hit — exit at the last close.                                  |
| <code>tp_timeout</code>  | In profit at the timeout, or the post-timeout breakeven target was hit.                           |
| <code>sl_timeout</code>  | The post-timeout swing stop was hit after the breakeven adjustment.                               |

## 8. The optional indicators layer (CVD & VWAP)

Two finders and the two VWAP risk scripts use an optional second dataset. `__init__.py` resolves it as a sibling folder under the same `parquet/{type}/{asset}/` directory, named by the `indicators_folder` parameter, and loads the matching `YYYY-MM-DD.parquet` per day. If the parameter is empty, the folder or daily file is missing, or the read fails, the day simply runs without indicators: the CVD finders disable themselves and the VWAP risk scripts return no trade. Only the consumed columns are read (with a full-read fallback for files missing any):

| Indicator column(s)                                  | Consumed by                                                                                     | For                                                                     |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>cumulative_delta</code>                        | both CVD-divergence finders + both CVD trail detectors + <code>build_cvd_change_baseline</code> | Pivot CVD readings and the z-score volatility denominator.              |
| <code>vwap_tick_{globex,rth}_std{2,3}_{up,dn}</code> | <code>vwap_tp_risk</code> , <code>vwap_trailing_risk</code>                                     | The $\pm 2$ -sigma / $\pm 3$ -sigma deviation-band targets (8 columns). |

Produced by `transforms/lm_advanced_indicators.py`. A third parameter, `big_trades_folder`, is declared for an upcoming feature but reserved — not yet consumed. Alignment uses a fast path: when the indicator file shares the candle index (the normal case) the RTH mask applies positionally; otherwise it falls back to a reindex.

## 9. The day-core cache

The single biggest practical speedup. Everything about a day that does NOT depend on parameters — the numeric bar arrays, the parsed JSON columns, the per-bar passive maxima, the raw indicator arrays — is a pure function of the two files on disk. The package therefore keeps each day's built `DayData` ("day core") in an in-process cache and reuses it across backtest runs.

| Property                  | Behaviour                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Location                  | RAM only — a dict on a holder module registered once in <code>sys.modules("__ivb_day_cache_store")</code> . Nothing is ever written to disk. The holder survives the backtester's plugin loader, which RE-EXECUTES the package <code>__init__</code> on every run (a plain module-level dict would be wiped each time).                                           |
| Key                       | <code>(candle_path, mtime_ns, size)</code> + the same triple for the indicator file (or None). Params are deliberately NOT in the key.                                                                                                                                                                                                                            |
| Invalidation              | Automatic: re-running a transform rewrites the parquet file, its mtime changes, the key no longer matches, and the day is rebuilt from disk on the next run. Days skipped by <code>skip_existing</code> keep their old mtime and stay validly cached. Stale entries linger harmlessly until the LRU cap evicts them (they are least-recently-used by definition). |
| Capacity                  | <code>_DAY_CACHE_MAX_DAYS = 600</code> (LRU eviction, oldest-used first). Measured payload: ~0.28 MB per fully-parsed enriched ES day → 307 days = 87 MB; the cap is a ~200 MB ceiling.                                                                                                                                                                           |
| Lifetime                  | The Python process. Closing/restarting Streamlit drops the cache; the first run after a launch is a cold run (~4 s over 307 ES days) that rebuilds it. Switching pages, params, strategies or datasets within a session does NOT drop it.                                                                                                                         |
| What changing things does | Changing params or strategy code: cache stays (all hits). Changing <code>indicators_folder</code> : new keys, one cold rebuild. Different candle dataset: cached separately alongside. Re-transformed files: exactly those days rebuild.                                                                                                                          |

Visibility

One console line per run: `[ivb cache] day-cores: 307 from memory, 0 built, 307 cached.`

## Cached vs. rebuilt every run

| Cached on the day core (param-independent)                                                                                                                                                                                                                                                                  | Rebuilt per run (param-dependent)                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OHLC / volume_delta_pct / buy+sell volume arrays; RTH index; warm-up positions; parsed <code>tick_volume</code> and <code>passive_orders</code> (lazy, at most once ever); per-bar max defended-side passive size (lazy); raw <code>cumulative_delta</code> and the 8 VWAP band arrays, already RTH-aligned | Rolling sell/buy baselines, passive baselines, CVD-change std (all depend on <code>absorption_baseline_window / tick_size</code> ); the volume profile (depends on <code>ib_minutes</code> ); breakout/retest, window masks, entries, trails, fills |

`process_day` begins every call with `day.reset_run_state()`, wiping the previous run's baselines before rebuilding under the current params — a cached day can never leak stale param-dependent state into a new run. The boundary is deliberate: everything inside the cache is a pure function of the files, nothing else, which is what makes the byte-identical guarantee hold on warm runs too.

## 10. Vectorized internals (what makes it fast)

The original processed bars through per-row DataFrame access (`.iloc[i]` materializes a mixed-dtype row object, ~60 microseconds) and re-parsed the same JSON strings in every finder and every trailing detector. The rewrite materializes a day once and hands out positional views:

### DayData (per day, cacheable)

- Numeric columns converted once at C level to float64 numpy arrays, RTH-masked (the RTH slice uses integer minutes-since-midnight, not per-bar time objects; no sliced DataFrame is ever created).
- `tick_volume` / `passive_orders` parsed ONCE per day into per-bar (`prices`, `a`, `b`) arrays kept in JSON document order — so every "first qualifying level" scan picks the same level the original dict-iteration found. The whole column parses as one JSON array (fast path) with whole-day batched numpy conversion; a per-bar fallback isolates malformed bars, and a dtype guard falls back to per-bar arrays if any value is non-integer (preserving the exact python types that end up in the notes). Missing / "{}" / unparseable values become None — the original per-access "no data" semantics.
- Parsing and the passive-max pass are lazy memoized properties: a day or a param set that never touches them never pays; once computed they live on the cached object.

### EntryWindow / TradeWindow (per trade attempt)

`EntryWindow` is the post-retest window ([breakout bar] + retest block) as a position array into the day; `TradeWindow` is the contiguous post-entry slice. Both precompute, once, the masks every consumer shares: `invalid` / `first_inv` (close through VAL/VAH), `confirm_any` (body + delta), `confirm_dir` (+ candle colour), `wick_frac` (defended wick fraction, -inf where range <= 0), plus direction-selected baseline gathers. Finders keep their sequential control flow (earliest-invalidation-wins, first-match scans, running `seen` lists) but iterate only over vector-prefiltered candidate bars using numpy scalars.

### Exact-equality invariants the rewrite preserves

- First-match scans over JSON dicts → document-order arrays + argmax.
- Floating-point summation order in the two-bar pair baseline → strided gather summed in the original loop order (python sequential sum).
- `json.loads` failure => "no data" → preserved at parse-once time (and per-bar in the fallback).
- Notes dict key order and python-native value types (json.dumps rejects numpy scalars) → note-building code kept verbatim; array values cast via `float()` / `.item()`.
- pandas rolling mean/std kept for all baselines → bit-identical floats (a cumsum rewrite would drift in the last ulp).
- Same-bar tie rules, flip resume position (AT the invalidation bar), stable earliest-entry tie-break (registry order) — all replicated exactly.

### I/O path

Candle reads request only the 9 consumed columns (of 11), indicator reads 9 of ~32, both with full-read fallbacks. A

sliding-window prefetch (2 reader threads, 4 files ahead) overlaps the next days' parquet reads with the current day's compute; days are consumed strictly in file order so the output is identical to a sequential loop. On warm runs the readers are idle — cache hits never touch disk (beyond an mtime stat per file).

## 11. Timing instrumentation

Every stage is wrapped in an accumulating timer (`_timing.timed(name)`), perf\_counter-based, stage-level — never per bar). `run()` resets the registry, and at the end prints ONE aggregated table — never per-day spam:

```
[ivb timing] wall 0.726s - accumulated sections (nested: children also count inside parents)
section                                total s    calls    ms/call    % wall
day:process_day                        0.651      307      2.119      89.7%
risk:vwap_trailing_risk                 0.364      158      2.303      50.1%
risk4:build_trailing_sl                 0.357      158      2.262      49.3%
risk4:trail:consecutive_absorption      0.098      158      0.622      13.5%
day:baselines                          0.098      304      0.321      13.5%
...
```

Sections nest (a child's time is also inside its parent), so the percentages are a map of where time goes, not a disjoint partition. On cold runs the `io:*` sections run on prefetch threads and can sum to more than wall time; `io:stall` is the actual time the main loop waited for a read. Section prefixes: `io:` reads/stalls · `day:` per-day stages (build\_core, daydata\_build, rth\_filter, indicators\_align, profile, baselines, cvd, breakout\_retest, window builds, process\_day, notes) · `entry:<finder>` per enabled finder · `risk:<script>` risk total · `risk4:trail:<detector>` per trailing detector, plus `risk4:build_trailing_sl` and `risk4:fill_sim`. If a future change makes something slow, the culprit section is immediately visible in the console.

## 12. Measured performance

All figures: 307 ES days (`ES_1m_advanced` + `ES_1m_indicators`), same machine, byte-identical outputs. "Original" is `strategies/ivb_model/` instrumented with the same wall-clock timers; "cold" is the first run of a process (cache empty), "warm" every run after.

| Configuration                                             | Original | Optimized cold | Optimized warm | Warm speedup |
|-----------------------------------------------------------|----------|----------------|----------------|--------------|
| risk 4, everything on (7 entries + 7 trails, trailing TP) | 26.8 s   | 3.9 s          | 0.73 s         | 37x          |
| risk 4, defaults ("1111100")                              | 22.7 s   | 3.9 s          | ~0.6 s         | ~38x         |
| risk 3 (vwap TP)                                          | 6.6 s    | 2.9 s          | 0.4 s          | 16x          |
| risk 1 / 2                                                | ~7 s     | 2.8 s          | 0.33 s         | ~21x         |

Per-component, everything enabled (original → optimized, warm):

| Component                                   | Original    | Optimized               | Speedup     |
|---------------------------------------------|-------------|-------------------------|-------------|
| risk 4 trailing machinery (per-trade total) | 18.65 s     | 0.49 s                  | 38x         |
| trail: two_bar_absorption                   | 7.31 s      | 0.12 s                  | 59x         |
| trail: absorption_delta / consecutive       | 2.7 / 2.4 s | 0.12 / 0.13 s           | 24x / 19x   |
| trail: passive size-only / passive wall     | 2.2 / 2.2 s | 0.016 / 0.014 s         | 137x / 155x |
| trail: CVD (both flavours)                  | 1.7 s       | 0.07 s                  | 23x         |
| entry finders (all seven together)          | 1.7 s       | 0.05 s                  | 17–80x each |
| baselines (rolling + passive x2 + CVD std)  | 3.1 s       | 0.10 s (warm)           | ~30x        |
| IVB volume profile                          | 0.32 s      | 0.07 s                  | 4.8x        |
| parquet reads + JSON parse + array build    | ~4.6 s      | 0 on warm runs (cached) | —           |

The remaining warm-run cost is the per-run rebuild (baselines + profile, ~0.17 s) plus the actual strategy logic (~0.5 s with everything on). The remaining cold-run cost is the parquet reads and the one-time JSON parse — inherent to touching the files at all.

## 13. Parameters

Defaults from `params.py`; `PARAM_SECTIONS` groups them in the backtester UI.

| Parameter                                                                                              | Default                     | Meaning                                                                                                     |
|--------------------------------------------------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------|
| ib_minutes                                                                                             | 30                          | IB duration in bars (typically 15 / 30 / 60).                                                               |
| trade_timeout                                                                                          | 999                         | Bars before the timeout/breakeven logic engages (999 = effectively off).                                    |
| max_flips                                                                                              | 4                           | Max direction flips per day after invalidation.                                                             |
| valid_entries                                                                                          | "1111100"                   | One on/off bit per finder, registry order (Section 5).                                                      |
| risk_script                                                                                            | 4                           | 1 = basic, 2 = zone SL, 3 = vwap TP, 4 = vwap trailing.                                                     |
| indicators_folder                                                                                      | "ES_1m_indicators"          | Sibling dataset for CVD + VWAP bands; empty = none.                                                         |
| big_trades_folder                                                                                      | "ES_big_trades"             | Declared, reserved for upcoming use.                                                                        |
| retest_window                                                                                          | 45                          | Max bars to wait for a retest after breakout.                                                               |
| entry_window                                                                                           | 25                          | Bars scanned for an entry after the retest.                                                                 |
| entry_after_absorption                                                                                 | 5                           | Bars scanned for a confirming candle after a signal.                                                        |
| absorption_baseline_window                                                                             | 20                          | Rolling window (bars) for every baseline.                                                                   |
| delta_threshold                                                                                        | 10.0                        | Min  volume_delta_pct  for a confirming candle.                                                             |
| body_threshold                                                                                         | 0.5                         | Min body/range ratio for a confirming candle.                                                               |
| wick_threshold                                                                                         | 0.4                         | Min defended-wick/range ratio (absorption_delta).                                                           |
| absorption_mult                                                                                        | 2.0                         | Wick-level volume >= this x baseline.                                                                       |
| consec_abs_n / _mult / _ticks /<br>consec_wick_threshold                                               | 2 / 2.0 / 4 / 0.4           | Consecutive absorption: candles required, multiplier, +-tick clustering tolerance, wick threshold.          |
| two_bar_wick_ticks / two_bar_abs_mult                                                                  | 8 / 2.0                     | Two-bar: max defended wick (ticks) on both bars; absorption multiplier for the merged pair.                 |
| passive_size_order_mult / _absorption_mult /<br>_wick_threshold                                        | 2.0 / 1.5 / 0.4             | Passive size-only: resting size multiple, absorption multiplier, wick threshold.                            |
| passive_wall_n / _mult / _ticks                                                                        | 3 / 2.0 / 8                 | Passive wall: orders needed, size multiple, +-tick clustering tolerance.                                    |
| cvd_pivot_k / cvd_min_separation /<br>cvd_max_separation / cvd_wick_tolerance_ticks /<br>cvd_min_score | 2 / 3 / 20 / 2 / 0.3        | CVD divergence (absorption flavour): fractal size, pivot separation bounds, price tolerance, min  z-score . |
| cvd_exh_* (same five)                                                                                  | 2 / 3 / 20 / 2 / 0.3        | Independent parameter set for the exhaustion flavour.                                                       |
| rr / sl_type                                                                                           | 1.0 / 0                     | basic_risk: reward multiple; 0 = VAL/VAH stop, 1 = swing.                                                   |
| zone_rr                                                                                                | 1.0                         | zone_sl_risk: reward multiple.                                                                              |
| sl_placement / vwap_std / vwap_session /<br>vwap_tp_mode                                               | 2 / 2 / "globex" /<br>"now" | vwap scripts: 1 = VAL/VAH stop, 2 = zone logic; target sigma; band session; frozen vs trailed target.       |
| trailing_entries / trailing_in_profit / late_trailing                                                  | "1111100" / 1 / 0           | Script 4: which signals may trail (bit order = valid_entries); breakeven-or-better filter; lag one signal.  |

|           |        |                                                                              |
|-----------|--------|------------------------------------------------------------------------------|
| tick_size | (auto) | Injected from ASSET_INFO by the backtester; in HIDDEN_PARAMS — no UI widget. |
|-----------|--------|------------------------------------------------------------------------------|

## 14. Output schema & the notes column

One row per trade with these **OUTPUT\_COLUMNS**. The strategy never stores a **ticks** column — the backtester derives ticks from **pnl\_points** via the asset's tick metadata.

| Column                   | Type      | Description                                                                                        |
|--------------------------|-----------|----------------------------------------------------------------------------------------------------|
| date                     | date      | Trade date (the day file).                                                                         |
| direction                | str       | Lowercase 'long' / 'short'.                                                                        |
| trade_type               | str       | Which finder fired (one of the seven names in Section 5).                                          |
| entry_time / exit_time   | Timestamp | Entry-bar open time / exit-bar time.                                                               |
| entry_price / exit_price | float     | Entry-bar open / the SL, TP, trailed level, or close at exit.                                      |
| sl / tp                  | float     | Stop / target levels used (sl keeps the ORIGINAL stop even when a trailed level exited the trade). |
| exit_reason              | str       | tp / sl / trailing_sl / eod / tp_timeout / sl_timeout.                                             |
| pnl_points               | float     | exit-entry (long), entry-exit (short), in points.                                                  |
| notes                    | JSON str  | Context + finder + risk keys, see below.                                                           |

Per-day context merges with finder-specific and risk-specific keys into one JSON string. A representative long via **absorption\_delta** under risk script 4:

```
{
  "breakout_time": "10:03", "retest_time": "10:11", "flip_count": 0,
  "ivb_high": 5102.25, "ivb_low": 5094.50,
  "poc": 5097.75, "vah": 5100.25, "val": 5095.50,
  // finder keys (absorption_delta):
  "absorption_time": "10:14", "abs_baseline": 118.4,
  "trigger_price": 5096.0, "trigger_volume": 412,
  // risk keys (vwap_trailing_risk):
  "tp_type": "tp_vwap_2", "escalated": false, "trail_count": 2,
  "trail1_time": "10:32", "trail1_type": "consecutive_absorption",
  "trail1_stop": 5099.5, "trail1_absorption_time": ["10:28", "10:31"], ...
  "trail2_time": "11:05", "trail2_type": "passive_wall", "trail2_stop": 5101.25, ...
}
```

Finder note keys: **absorption\_delta** → **absorption\_time**, **abs\_baseline**, **trigger\_price**, **trigger\_volume**; **consecutive** → the same with **absorption\_time** as a list; **two\_bar** → **absorption\_time** (pair), **two\_bar\_baseline**, **trigger\_\***; **passive size-only** → **absorption\_time**, **passive\_price**, **passive\_size**, **passive\_count**; **passive wall** → **wall\_levels**, **wall\_times**, **wall\_count**; CVD finders → **cvd\_pivot1/2\_time/price/cvd**, **cvd\_score**, **cvd\_change\_std**.

## 15. Data requirements (inputs)

IVB requires enriched one-minute candles from `transforms/1m_advanced.py`, currently available for ES / NQ only (the enriched Databento window). Plain OHLCV is not sufficient. Each day is one `YYYY-MM-DD.parquet` with a tz-aware America/New\_York index. The optimized package reads ONLY the columns below (column-pruned parquet read with a full-read fallback):

| Column                                | Required      | Used for                                                              |
|---------------------------------------|---------------|-----------------------------------------------------------------------|
| open high low close                   | yes (float64) | IB range, breakout/retest, bar structure, SL/TP.                      |
| buy_volume sell_volume                | yes           | The per-tick rolling absorption baseline + the two-bar pair baseline. |
| volume_delta_pct                      | yes           | Confirming-candle delta threshold; two-bar pattern delta.             |
| tick_volume (JSON {price:[buy,sell]}) | yes           | Volume profile + absorption grading.                                  |

|                                            |     |                                                           |
|--------------------------------------------|-----|-----------------------------------------------------------|
| passive_orders (JSON {price:[size,count]}) | yes | The two passive finders + passive baseline.               |
| volume, volume_delta                       | no  | Present in the files but not read by this package at all. |

The optional indicators dataset (Section 8) adds `cumulative_delta` and the 8 `vwap_tick_*` band columns; without it the model still trades on the five order-flow finders and the two point-based risk scripts.

## 16. Equivalence guarantee & how to re-verify

The optimized package was verified against the untouched original with

`pandas.testing.assert_frame_equal(check_exact=True)` — every column, every dtype, including exact `notes` string equality — over all 307 ES days for 12 parameter configurations: risk scripts 1–4, all-entries-on, both `vwap_tp_modes`, `sigma-3` + VAL/VAH placement, late trailing + in-profit off, a 30-bar timeout (exercising the breakeven tails), reduced windows with flips, loosened thresholds, and a mixed entries-vs-trailing bit pattern. The warm-cache path was verified the same way (repeat runs in one process, and repeat loads through the backtester's own `exec_module-per-run` loader).

**To re-verify after future edits:** put `strategies/` on `sys.path`, run each config through `ivb_model` (ground truth; cache the frames with `to_pickle`) and through `ivb_model_optimized`, and compare with `assert_frame_equal(..., check_exact=True)`. Any real logic change is expected to break equality — that is the point: the twin packages should only ever diverge deliberately.

## 17. Known limits & roadmap

- **Session times are hardcoded** 09:30–16:00 NY — correct for equity-index futures, wrong for asset classes with different RTH.
- **Enriched data coverage is thin** — `tick_volume` / `passive_orders` exist for ES/NQ over roughly one year; walk-forward validation needs multi-year enriched data.
- **Baseline warm-up** — baselines need a full 20-bar window from 09:35, so the earliest post-open candles cannot be graded.
- **Flip conservatism** — a re-detected breakout in the wrong direction yields no trade rather than a forced entry.
- **No higher-timeframe context** — the model operates purely on one-minute bars.
- **Cache lifetime is the process** — a Streamlit restart costs one ~4 s cold run. A disk-backed day-core cache would cut the cold start to ~1 s if that ever matters.
- **Python-only floor** — the remaining warm-run cost is near the numpy floor; further gains would need batched segmented scans, numba/Cython, or multiprocessing (all deliberately out of scope).

| Capability                                                                                    | Status |
|-----------------------------------------------------------------------------------------------|--------|
| Peak-based volume profile + POC refinement inside the value area                              | done   |
| Absorption+delta, consecutive, two-bar, passive size-only, passive wall entries               | done   |
| CVD-divergence absorption & exhaustion entries (indicator-gated, independent params)          | done   |
| Four self-contained risk scripts (basic / zone / VWAP-band / VWAP-band + trailing stop)       | done   |
| <code>trade_type</code> column + per-finder, per-risk and per-trail notes; direction flipping | done   |
| ASSET_INFO tick injection via <code>HIDDEN_PARAMS</code> (30+ instruments)                    | done   |
| Vectorized day context, day-core cache, gated baselines, stage timers, pruned/prefetched I/O  | done   |
| Byte-identical equivalence harness vs. <code>ivb_model</code> (12 configs, full ES dataset)   | done   |
| Per-asset session times                                                                       | todo   |
| Consume <code>big_trades_folder</code> dataset                                                | todo   |
| Min-risk threshold: relocate SL / extend TP when POC too close to VAH/VAL                     | todo   |
| Disk-backed day-core cache (instant cold start)                                               | todo   |
| Walk-forward validation on multi-year enriched data                                           | todo   |

Older performance / Monte-Carlo tables described the legacy single-entry model and remain removed pending a fresh

backtest. With seven finders and four risk scripts, results should be broken down by trade\_type and risk\_script before any numbers are quoted.